

DDPAGENT for Demand-Driven Data Preparation via Agentic Action Allocation and Operator-Grounded Execution

Zekai Qian

Harbin Institute of Technology, China
qzk010728@gmail.com

Xiaoou Ding*

Harbin Institute of Technology, China
dingxiaoou@hit.edu.cn

Hongzhi Wang

Harbin Institute of Technology, China
wangzh@hit.edu.cn

ABSTRACT

Agentic data systems can invoke many data-cleaning operators, but they still lack a task-level mechanism for deciding which intervention a downstream task needs. In machine-learning pipelines, repairing a cell, deleting a record, replacing a value, applying all operators, or preserving the dirty input may lead to different utility under different metrics, data states, and costs. We present DDPAGENT, a demand-driven data preparation framework that separates action-family allocation from operator-grounded execution. Given a task demand, DDPAGENT trains a hierarchical allocator without additional human labels for action selection, assigns high-level cleaning actions such as NO-OP, REPAIR, DELETE, and REPLACE to candidate units, and compiles the selected actions into executable operator workflows with provenance. Downstream validation then chooses among conservative rollback to No-op, selective action execution, and intensive FULLOPS execution. The paper makes four contributions. It introduces a task-demand formulation that treats preservation and full execution as valid agent decisions, connects zero-new-label action allocation with action-indexed operator networks, records action, workflow, operator, and operation provenance, and evaluates the design on five native-error datasets plus 40 mixed-error injected settings. The injected task tables have average cell-level error rate 1.58%, average record-level error rate 17.78%, and maximum record-level error rate 50.90%. Against named fixed cleaning methods, DDPAGENT has the highest average utility and is ranked first or second on almost every native and injected aggregate row. The results show that data preparation is not monotone in cleaning strength. Some tasks demand selective repair, some demand broad operator execution, and some are best served by preserving the data after validation.

VLDB Workshop Reference Format:

Zekai Qian, Xiaoou Ding, and Hongzhi Wang. DDPAGENT for Demand-Driven Data Preparation via Agentic Action Allocation and Operator-Grounded Execution. VLDB 2026 Workshop: ADS 2026: The Joint Workshop on Agentic Data Systems and Data-Centric AI (The 1st ADS & 3rd DATAI).

1 INTRODUCTION

Data-centric AI has made data quality a first-order design problem rather than a preprocessing detail [7, 8]. Dirty, missing, inconsistent,

or biased records can dominate downstream model behavior even when the model architecture is unchanged [1, 11, 12]. Existing data cleaning research has developed many ways to detect and repair erroneous cells, including probabilistic inference, contextual repair, rule reasoning, and mixed-error workflow coordination [14, 15, 17, 21]. These methods provide concrete operators for data-centric systems, but they do not by themselves determine which intervention a downstream task needs.

The missing layer is a task-level controller that decides whether the current task demands repair, deletion, replacement, preservation, or full operator execution before an operator workflow is compiled. This decision is not monotone in the amount of cleaning. Repairing every suspicious cell may increase cell-level correctness but can erase rare patterns or overfit to an imperfect reference. Deleting all suspicious rows may remove harmful records but can collapse sample diversity. Automatic replacement is cheap and scalable, yet it can propagate systematic errors when the value estimator is misaligned with the target metric. Conversely, leaving data unchanged is not always a weak baseline. In a governed workflow, it is a rollback decision when validation finds limited data-side gain. Full operator execution is the opposite regime, indicating that broad preparation has higher validation utility than selective pruning.

Consider a user who wants to train a downstream classifier over a dirty tabular dataset containing typographical errors, missing values, outliers, and dependency violations. A conventional cleaning pipeline asks which cleaner should be run and in what order. A task-driven user asks which records or cells are worth changing for the downstream model. A typo in a high-impact feature may need repair. A suspicious row with implausible values may need deletion. A missing value in a low-sensitivity feature may be safely replaced by an automatic estimator. A rare but valid boundary example should be preserved even if a detector marks it as suspicious. If validation shows that none of these edits improves the task, the agent should return to the NO-OP regime. If most workflow operations are aligned with the task, it should allow FULLOPS instead of pruning useful improvements. Figure 1 illustrates the instance-level hierarchy. The agent first selects an action family, compiles an ordered operator workflow under that family, and emits concrete data operations. The operation notation records the predicate scope, target attribute, candidate value, and originating operator when those fields apply.

This example exposes three challenges for agentic data preparation.

(1) Task demand cannot be inferred from repair correctness alone. The same detected issue may require different actions under different downstream metrics, models, budgets, or validation splits. A controller must learn intervention intensity from task feedback instead of from a fixed ordering of suspicious cells.

*Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment.

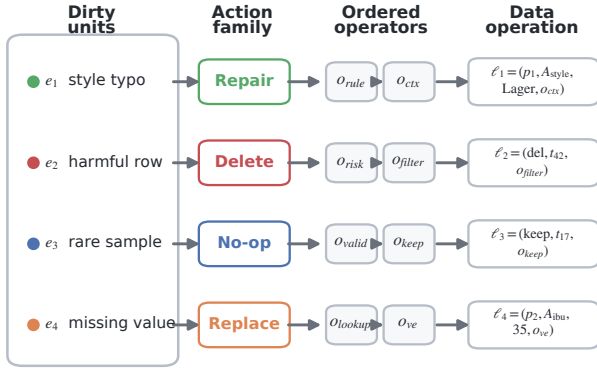


Figure 1: A cleaning instance of DDPAGENT. The agent first selects an action family, then compiles an ordered operator workflow and emits concrete data operations. In the cleaning setting, operators are cleaners, filters, preservers, or value estimators.

(2) Action allocation and operator execution live at different granularities. Choosing REPAIR, DELETE, REPLACE, or NO-OP specifies the intent of a preparation decision, but it does not determine which rules, contextual signals, estimators, or filters should be ordered into a workflow, nor which concrete value update or row operation should be emitted.

(3) Feedback must govern both regimes and operators. A prepared dataset should be treated as a candidate version. Validation may accept selective execution, roll back to NO-OP, or promote FULLOPS. The same feedback should also update which actions and operators are trusted later, without requiring new manual labels only to train the allocation policy.

DDPAGENT uses a controller and executor loop to address these challenges. An action family captures the semantic intent of an intervention, such as NO-OP, REPAIR, DELETE, or REPLACE. An operator is an executable procedure under that intent, such as a contextual cleaner, a rule-based repairer, a value-estimation routine, or a filtering operator. The controller allocates action families according to task demand. The executor grounds each selected action as an ordered operator workflow and materializes the resulting data operations. Downstream validation selects the final governance regime and feeds utility back to the controller. The current cleaning instantiation uses a learned action allocator trained with zero additional human ground-truth acquisition. In this paper, zero-new-label training means that the policy can use existing operator outputs, cached cleaned versions, and downstream validation feedback, but it does not request new manual labels just to learn which action should be taken.

DDPAGENT is a system-level framework rather than a new low-level cleaner. Its contribution is the reusable interface between task demand and operator execution. The system decides whether the task calls for conservative preservation, selective intervention, or full operator execution, then grounds the selected regime in ordered operator workflows and auditable data operations.

We study demand-driven data preparation as governed control over action families, regimes, and reusable operators, and make four contributions.

(1) A demand-driven preparation formulation. We formulate data preparation around task demand rather than around a fixed cleaning objective. The control boundary is an action family or workflow regime. It includes NO-OP rollback when the data side offers limited measurable gain, selective action allocation when only part of the emitted data operations is useful, and FULLOPS when broad operator execution is beneficial. This formulation makes preservation and full execution accountable decisions of the agent rather than weak baselines or external competitors.

(2) A zero-new-label controller and executor architecture. We design a two-level agent in which a hierarchical action allocator learns intervention intensity without requesting additional human labels, and an operator-grounded executor compiles the selected action into ordered operator workflows and concrete data operations. The allocator uses downstream utility, budget, detector, model, and execution-history signals. The executor uses action-indexed operator registries so that each action family can own its own compilation network.

(3) An operator-grounded governance trace. DDPAGENT records action-level, workflow-level, operator-level, and operation-level provenance for each prepared data version. The trace supports rollback, promotion to FULLOPS, operator-weight updates, and reuse across future cleaning demands. We evaluate the interface through tabular cleaning, where the implemented action families are preservation, repair, deletion, and replacement.

(4) Empirical evidence for demand-driven intervention. We evaluate five native-error datasets and 40 mixed-error injected settings against fixed cleaners, DDPAGENT regimes, and oracle diagnostics. The injected results are reported as per-dataset averages over all eight perturbation rates, and the injected task tables have average cell-level error rate 1.58%, average record-level error rate 17.78%, and maximum record-level error rate 50.90%. The final DDPAGENT governance output has the highest average utility among named fixed-cleaner outputs and is first or second on almost all aggregate rows. The selective branch improves or ties FULLOPS on most rows with average gain 0.0254, while the row where FULLOPS is better identifies a task that benefits from broad operator execution. The reported rows never fall below NO-OP after validation. The action traces show that native Tax is deletion-heavy, native Beers and Flights are replacement-heavy, and injected Tax shifts toward a NO-OP/DELETE mixture. Together, these cases support the paper’s central design choice. Reliable preparation requires choosing the demanded governance regime, compiling the corresponding operator workflow, and materializing the resulting data operations.

2 RELATED WORK

We organize prior work by the decision boundary it owns. Fixed cleaning systems decide how to produce a cleaned table. Workflow systems decide how to coordinate multiple operators. Task-aware preparation systems use downstream feedback but usually decide which sample, pipeline, or repair strategy to clean. DDPAGENT moves the decision boundary one level higher. It first decides the

demand action family and governance regime, then grounds that decision in reusable operators.

Fixed data cleaning operators. Classical cleaning work studies how to detect and repair data errors using data-level evidence [1]. Constraint-based systems reason over denial constraints, dependencies, and violation structures to search for consistent repairs [3, 23]. Scalable cleansing systems organize rules and operators into executable workflows for large data [10]. Probabilistic and Bayesian systems integrate constraints, co-occurrence statistics, and uncertainty to infer likely cell repairs [20, 21], while optimal-transport and conditional-independence approaches target more specific statistical violations [19]. Learning-based, retrieval-based, and language-model-assisted systems further expand the signal space by using contextual representations, transferred correction evidence, clean corpora, or LLM reasoning [4, 5, 14, 15, 18, 26]. These methods provide concrete repair, detection, imputation, and filtering operators. Their decision boundary, however, is usually a repaired cell, a repaired table, or a fixed cleaning pipeline optimized by data-level criteria. They do not decide whether the downstream task should preserve, repair, delete, replace, or fully execute operators on a particular unit.

Operator coordination and mixed-error workflows. Real tables rarely contain a single isolated error type. A repair produced for one attribute may become the source evidence for another operator, and two operators may propose conflicting values for the same target attribute. Existing systems study multi-signal integration, dependency-aware execution, and workflow optimization [10, 16, 21, 23, 24]. This line of work motivates the executor in DDPAAGENT. Operators should expose their scope, target, cost, output, and provenance so that a workflow can reason about dependencies and conflicts. The remaining gap is the control boundary above the workflow. Coordinating cleaners answers how a repair workflow should run once repair is desired. It does not by itself answer whether the task demands repair at all, or whether deletion, low-cost replacement, preservation, rollback, or full operator execution is the better governance decision.

Task-aware data preparation. Another line connects data cleaning or preparation with downstream machine learning. ActiveClean prioritizes records for cleaning under model training feedback [12], BoostClean searches over detector and repair combinations for model performance [11], and GoodCore studies data-efficient learning over incomplete data [2]. CPClean focuses on certain predictions over incomplete information [9]. Context-aware and pipeline-level systems construct or optimize preparation plans for ML tasks [6, 16, 24]. These systems establish the central premise that downstream utility should guide data preparation. Their controls are nevertheless different from ours. They usually clean selected records, choose a pipeline, optimize an order, or target a specific prediction setting. DDPAAGENT makes the per-unit action family explicit and treats No-op rollback, selective execution, and FULLOps promotion as governed outcomes rather than as external baselines.

Data-centric AI and extensible preparation actions. Data-centric AI argues that improving data can be as important as changing models [7, 8], and data-quality-aware MLOps emphasizes that data interventions must be tracked and evaluated in operational ML

workflows [22]. Related preparation operators include uncertainty-driven imputation and imputation-order optimization [25, 30], data valuation and selection [13, 28], and augmentation operators such as mixup and CutMix [27, 29]. We do not evaluate these non-cleaning actions in this paper. They clarify the intended boundary of DDPAAGENT. A data-governance agent should register new action families only when their units, admissible operators, costs, outputs, and provenance are explicit. The evaluated instance is tabular cleaning, where this boundary is instantiated by preservation, repair, deletion, and replacement.

3 PROBLEM FORMULATION

We formalize the layer that sits between a downstream task and a library of preparation operators. Let D be a dirty training dataset for task T with evaluation metric U . The user or task interface may also provide a candidate model set $\mathcal{M}$, such as random forest, SVM, gradient boosting, or a task-specific estimator. A data profiler, detector, cached prepared version, or prior workflow output produces candidate preparation units $E = \{e_1, \dots, e_n\}$. A unit can be a cell, a row, a feature group, or a structured violation. In the cleaning instance, it is usually a detected cell-level error. The agent observes a state s_i for each unit, chooses an action family a_i , and asks an executor to realize the selected family by ordering operators and emitting concrete data operations. The output is a workflow W , a prepared dataset D_W , and a provenance trace τ_W .

For example, in the Hospitals task used later, T is measure-code classification, U is accuracy, E contains candidate cells or violations obtained from detectors and cached cleaning outputs, and $\mathcal{A}$ contains preservation, repair, deletion, and replacement. In the Tax-10K task, T is rate regression on a clustered 10K subset, so a suspicious record may be more useful as a deletion candidate than as an individual cell repair. The formulation below keeps both cases in the same interface. The task demand changes, while the allocator and executor still communicate through action families and operator registries.

We introduce the interface in the order that information moves through the system. A task demand first specifies what the downstream workflow cares about. The action family then states the intended intervention for a unit. After the selected actions are executed, the governance regime decides which prepared data version should be committed. Operator grounding and the demand-driven policy connect these objects to executable workflows and learned control.

DEFINITION 1 (TASK DEMAND). *A task demand is a tuple*

$$q = (T, U, \mathcal{M}, D, E, \mathcal{A}, \Omega, B, \mathcal{P}),$$

where T and U define the downstream task and metric, $\mathcal{M}$ is the optional candidate model set, E is the preparation-unit set, $\mathcal{A}$ is the action-family space, $\Omega = \{\Omega_a \mid a \in \mathcal{A}\}$ is the action-indexed operator registry, B is the human or execution budget, and $\mathcal{P}$ contains policy constraints such as whether deletion is allowed. Evidence sources such as detector scores, rule violations, cached cleaned versions, validation outputs, or prior execution traces are encoded in the state s_i . Governance regimes are defined separately because they are commit decisions after execution rather than fields of the user demand.

The tuple is deliberately wider than a dirty table and a detector output. It makes the downstream objective, candidate models, action space, operator registry, and governance constraints visible before execution. This is necessary because the same suspicious cell can be handled differently when the budget, task metric, or admissible operators change.

DEFINITION 2 (ACTION FAMILY). An action family $a \in \mathcal{A}$ denotes the semantic intent of a preparation decision. It is not an executable program. In the cleaning instantiation, the action space is

$$\mathcal{A}_{\text{clean}} = \{\text{NO-OP}, \text{REPAIR}, \text{DELETE}, \text{REPLACE}\},$$

corresponding to preservation, correction, removal, and low-cost substitution. The action space is an interface rather than a closed list. A system can add a new family by specifying its preparation units, state features, admissible operators, cost model, and provenance schema. For example, a data-augmentation family would not reuse the repair operators directly. It would register augmentation operators that expose their generated samples, affected labels or features, cost, and validation feedback through the same Ω_a interface. We evaluate the cleaning instance, but the controller and executor communicate through this extensible action and operator boundary.

The action family is local to a preparation unit. A workflow still needs a dataset-level decision after execution, because a selective candidate may fail validation, while a complete all-operator workflow may be better than selective pruning. This motivates a separate regime variable.

DEFINITION 3 (GOVERNANCE REGIME). A governance regime $\rho \in \mathcal{G}$ denotes the workflow-level decision after action execution and validation. In the current instantiation,

$$\mathcal{G}_{\text{clean}} = \{\text{rollback}, \text{selective}, \text{fullops}\}.$$

The rollback regime returns the current data version and corresponds to dataset-level NO-OP when edits do not improve utility. The selective regime commits only the data operations emitted by action-allocated workflows. The FULLOPS regime commits the complete all-operator workflow output when broad operator application is beneficial.

A regime says which data version should be committed, but it still does not specify what the executor emits. We first define the operation record that appears in the workflow and provenance trace.

DEFINITION 4 (DATA OPERATION). A data operation ℓ is the auditable edit or preservation record emitted by an action workflow for a preparation unit. It records the selected action family, the scope of affected tuples or cells, the target attribute when applicable, the proposed value when applicable, the originating operator, execution cost, and commit status. In repair or replacement, we write

$$\ell = (a, p, A_t, v', o, c_\ell),$$

where p is the predicate scope, A_t is the target attribute, v' is the candidate value, and o is the operator that generated the value. When the action and cost are clear, we use the shorter form $\ell = (p, A_t, v', o)$. A deletion operation records a scoped removal such as $\ell = (\text{DELETE}, p, o, c_\ell)$, and a NO-OP operation records intentional preservation of the unit. This object is the unit that can be committed, pruned, rejected, or traced back to an operator.

With data operations in place, the next definition gives the executor contract that maps an action family to an ordered operator workflow.

DEFINITION 5 (OPERATOR GROUNDING). For each action family a , Ω_a denotes the concrete operators that can realize it. Each operator exposes a scope, input requirements, source attributes, target attributes, cost, output type, and provenance fields. The grounding function $g(e_i, a_i, \Omega_{a_i})$ does not choose a single operator. It constructs or reuses an action-specific operator network, orders applicable operators, generates candidate operations ℓ , and appends accepted operations to workflow W .

Finally, the controller is the policy that chooses among these action families under the task demand.

DEFINITION 6 (DEMAND-DRIVEN POLICY). The allocator is a policy $\pi_\theta(e_i, s_i, q) \rightarrow a_i$ that maps an issue, a task-aware state, and a demand specification to an action family. The policy is demand-driven because the same issue may receive different actions under different downstream metrics, budgets, or operator availability.

The optimal tuple $(\rho^*, \pi^*, W^*, M^*)$ maximizes downstream utility after execution while accounting for human and execution costs.

$$\begin{aligned} \max_{\rho, \pi, W, M} \quad & U(M, T, D_{\rho, \pi, W}) \\ & - \lambda C_{\text{exec}}(\rho, W) - \gamma C_{\text{human}}(\rho, W) \\ \text{s.t.} \quad & \rho \in \mathcal{G}, \pi \in \Pi, \\ & W \in \mathcal{W}, M \in \mathcal{M}, \end{aligned} \tag{1}$$

subject to the allowed action space, operator availability, mutual exclusivity for each preparation unit, and any user-specified budget or policy constraints. If $\mathcal{M}$ contains a single model, Equation 1 reduces to the fixed-model setting. In the cleaning instantiation, human cost is especially important because ground-truth repair is expensive and often unavailable at deployment time. Zero-new-label action-allocation training corresponds to the regime where the allocator does not request new human labels to learn the policy, even though it may spend computation and reuse existing operator-generated evidence.

This formulation differs from operator-only cleaning. A fixed cleaner directly searches for a repair D' that improves data-level correctness. DDPAGENT searches over action families, operator realizations, and workflow regimes with respect to downstream utility. Downstream validation adds a governance layer around this objective. When a candidate execution lowers the validation utility relative to the reference data, the final decision can roll back to NO-OP. When the complete all-operator workflow is consistently better than selective pruning, the system can expose FULLOPS as the appropriate execution regime. Neither outcome is a failure case. Rollback means that the data side offers limited measurable gain for the current task, while FULLOPS means that the task benefits from broad operator application.

4 DDPAGENT FRAMEWORK AND EXECUTION LOOP

Figure 2 shows the four-block framework. The section follows the execution order of the system. Step 1 converts a downstream request into a typed demand. Step 2 trains an action allocator over

the candidate units without requesting new manual labels for policy learning. Step 3 executes the learned actions by indexing into action-specific operator registries, ordering the operators, and committing concrete data operations. Step 4 records action, workflow, operator, and operation provenance and feeds downstream validation back to both the allocator and the operator weights. The bottom feedback edge is downstream validation rather than an independent fifth module. It evaluates each prepared data version and can accept the selective candidate, roll back to the reference data, promote full operator execution, or update the allocation policy with the observed utility change.

4.1 Step 1: Demand Specification

The first step converts a downstream request into the typed demand tuple q . This input contract is more specific than a dirty table. It includes the downstream task and target column, the validation metric, optional candidate downstream models, candidate preparation units, action families, action-indexed operator registries, budget, and policy constraints. In the current cleaning instance, the candidate units are detected cells and additional cells suggested by differences between dirty data and cached cleaning outputs. The evidence sources used for state encoding, denoted by $\mathcal{S}$ in the algorithms, include detector confidence, rule violations, feature importance from the downstream model, cached repair values, and execution histories. The operator registry includes cleaning operators, value estimators, row filters, and pass-through operators.

Each operator is registered by a small schema containing source scope, target scope, input requirements, output type, cost, and provenance fields. For example, a repair operator may expose $A_s \rightarrow A_t$, a detector function, and a value generator. A delete operator exposes a row scope and a retention constraint. A replacement operator exposes a candidate-value source and admissible domain. This contract lets the allocator reason over action families without knowing the internal implementation of every operator, and lets the executor later build action-specific operator networks. It is also the extension mechanism of the framework. Adding an action family requires adding Ω_a and the corresponding state/provenance fields, not rewriting the allocator around one concrete tool. In the experiments, the tuple is produced by configuration and dataset metadata. The method only assumes that later decisions are conditioned on this structured demand.

The tuple also prevents a common ambiguity in tool-using data agents. A request such as “prepare the hospital table for measure-code prediction” is not executable until it is grounded into a target column, a validation split, allowed actions, candidate units, and operator registries. Once these fields are explicit, a cleaning operator, a row filter, and a value estimator can all be considered by the same controller without being treated as interchangeable tools.

4.2 Step 2: Zero-New-Label Action Allocation Training

The second step trains a task-aware action allocator without requesting additional human ground-truth labels. Zero-new-label here means zero new manual annotation for learning the allocation policy. The allocator may still use computation, detector outputs, cached dirty/cleaned versions, operator-generated candidates, and

validation feedback that already exist in the data governance environment. When candidate downstream models are provided, the system first selects or fixes the evaluation model used by the allocator. A practical choice is to prefer models that are both competent on the task and responsive to data preparation, because a model whose metric does not change under any preparation action gives weak reward signals.

Training is cast as a sequential decision problem over E . Each candidate unit is represented by a state vector rather than by its raw value alone. In the cleaning instance, the state vector contains local signals such as issue type, column type, detector confidence, feature importance, boundary distance, and column error rate. It also contains global data-quality signals such as sample retention and distributional retention, budget signals such as remaining human budget and remaining issue ratio, and execution-history signals such as whether an operator candidate exists and how similar operators behaved previously. The policy then learns a mapping from state and demand to an action family:

$$\pi_\theta(e_i, s_i, q) \rightarrow a_i \in \{\text{No-op}, \text{REPAIR}, \text{DELETE}, \text{REPLACE}\}.$$

The action family is intentionally coarser than an operator workflow. REPAIR means that a correction is demanded. It does not yet decide which rule, dependency, or contextual cleaners should be ordered, nor which candidate value should be committed. REPLACE means that a lower-cost estimate is acceptable. It does not hard-code which lookup or estimator should run first. DELETE means the issue scope is more harmful than useful under the task demand. No-op is a first-class decision when the detector is uncertain, the task is tolerant, or intervention would remove useful boundary cases.

The current allocator uses a hierarchical Dueling Double DQN design. Stage 1 performs cost-free triage and selects among No-op, DELETE, and *enter intervention*. Stage 2 is triggered only for intervention candidates and selects between low-cost replacement and costly repair. This prevents a flat action space from degenerating into always repairing whenever a reference value is available. Each stage has its own value and advantage heads,

$$Q_j(s, a) = V_j(s) + A_j(s, a) - |\mathcal{A}|^{-1} \sum_{a'} A_j(s, a'),$$

where $j \in \{1, 2\}$ denotes the stage. Target networks and replay buffers are maintained separately, because Stage 1 mainly learns whether intervention is needed, while Stage 2 learns how much cost the intervention should spend.

The reward combines three signals. A shaping reward gives dense guidance from issue priority, feature importance, detector confidence, deletion risk, and budget slack. A periodic proxy reward measures the change in downstream validation utility after a batch of decisions, subtracting cost for human repair. A terminal reward compares the final prepared candidate against the baseline data version while penalizing lost samples and spent human budget. If the remaining budget is exhausted, repair is automatically downgraded to a zero-cost replacement or No-op action according to policy constraints. This training procedure makes action allocation a task-conditioned controller rather than a fixed ranking of suspicious cells.

Algorithm 1 summarizes the training loop. The key point is that the environment is built from existing data versions and operator

DDPAGENT: Demand-Driven Data Preparation Agent

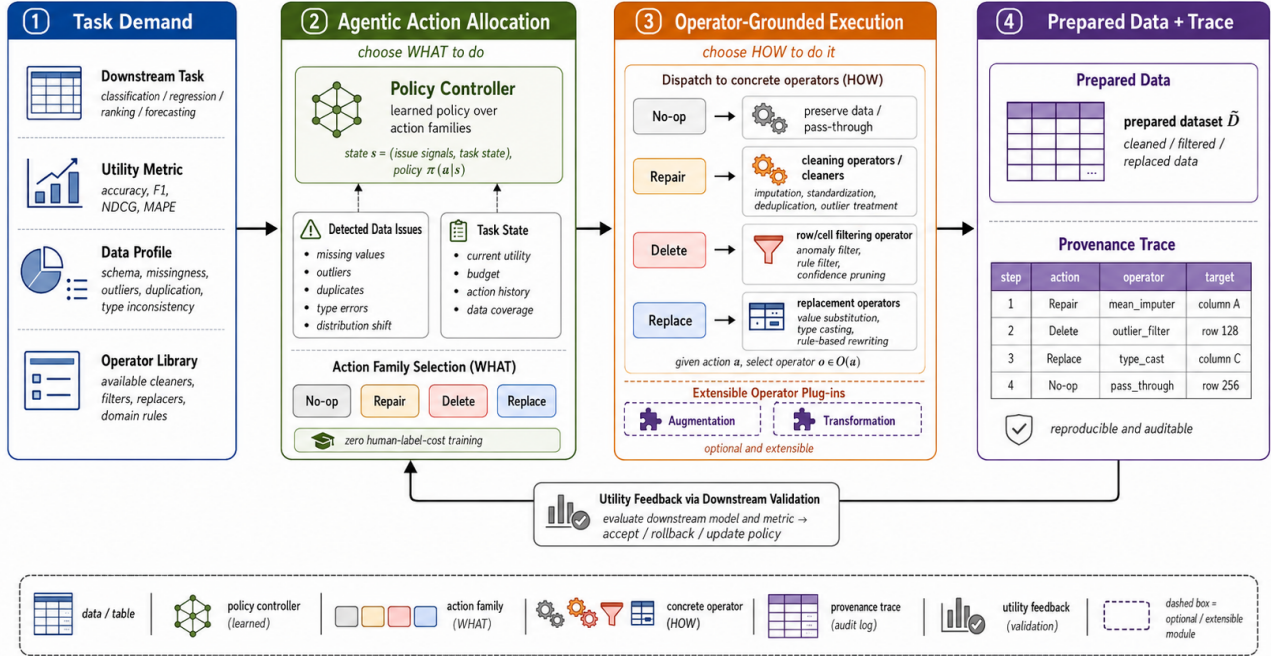


Figure 2: DDPAGENT framework. A task demand is converted into action-family allocation, grounded as ordered operator workflows, and materialized as prepared data with action, workflow, operator, and operation provenance. The bottom edge denotes downstream validation feedback, which can accept selective execution, roll back to No-op, promote FULLOps, or update the allocation policy.

evidence, not from newly requested manual labels. The allocator observes how candidate actions change downstream validation utility and learns a cost-aware policy over action families.

A typical decision illustrates the two-stage structure. If a candidate unit lies in a low-importance column with weak detector confidence, Stage 1 can select No-op and avoid unnecessary execution. If the unit is high-impact, Stage 1 enters intervention. Stage 2 then chooses REPAIR when the evidence store contains reliable correction evidence and the remaining budget allows it, or REPLACE when the task can tolerate a lower-cost estimate. This is why the policy is trained over action families rather than concrete cleaners. The allocator learns how much intervention the task demands before the executor orders operators and emits the concrete operation.

4.3 Step 3: Inference-Time Action Execution

At inference time, the learned allocator assigns action families to candidate units for a new demand. The executor then grounds each action through an action-specific operator network G_a . In the cleaning instantiation, No-op is grounded as traceable pass-through, REPAIR as an ordered repair workflow, DELETE as row- or cell-scope filtering, and REPLACE as an ordered value-estimation workflow. This division is the core of the data governance agent. The allocator decides what action the task demands, while the

executor decides how the selected action is realized by ordering the current operator pool and committing concrete operations.

The executor also supports workflow-level regimes. The default regime is selective execution, where only the operations emitted by action-allocated workflows are committed. The No-op regime is a conservative rollback path selected when validation shows that editing does not improve the current task. The FULLOps regime materializes the complete all-operator workflow output without selective pruning. It is useful when the downstream task benefits from broad application of the available operator pool. Reporting these regimes separately is not a comparison between DDPAGENT and external alternatives. It exposes the three outcomes that a data governance agent may choose under different data states.

Algorithm 2 gives the inference procedure. The policy only selects action families. The executor then indexes into the corresponding operator registry, builds an action-specific operator graph, orders applicable operators, searches executable operations under that graph, and lets downstream validation choose the final governance regime.

Each action family can have its own compilation network. For REPAIR, the network is built over heterogeneous cleaning signals. Each repair operator is represented as $o = [\text{detect}_o, f_o]$ from A_s to A_t , where A_s is the source scope, A_t is the target scope, detect_o estimates whether a tuple remains erroneous, and f_o generates

Algorithm 1 Zero-New-Label Action Allocation Training

Require: Demand q , dirty data D , units E , signals S , operators Ω , validation split, episodes N

Ensure: Action allocator π_θ

```
1: Build evidence store  $C$  from cached data versions, detector outputs,
   and operator candidates
2: Select or fix downstream evaluator  $M \in \mathcal{M}$  for utility  $U$ 
3: Initialize two-stage Q networks  $Q_1, Q_2$ , targets  $\bar{Q}_1, \bar{Q}_2$ , replay buffers
    $\mathcal{B}_1, \mathcal{B}_2$ 
4: for episode = 1 to  $N$  do
5:    $D^{(0)} \leftarrow \text{SAMPLEVERSION}(D, C)$ ,  $B_{rem} \leftarrow B$ , and  $\tau \leftarrow \emptyset$ 
6:   for each preparation unit  $e_i \in E$  do
7:      $s_i \leftarrow \text{ENCODE}(D^{(t)}, e_i, q, S, B_{rem}, \tau)$ 
8:      $z_i \leftarrow \text{EPSGREEDY}(Q_1, s_i, \epsilon)$  over  $\{\text{NO-OP}, \text{DELETE}, \text{intervene}\}$ 
9:     if  $z_i = \text{intervene}$  then
10:       $a_i \leftarrow \text{EPSGREEDY}(Q_2, s_i, \epsilon)$  over  $\{\text{REPLACE}, \text{REPAIR}\}$ 
11:      if  $a_i = \text{REPAIR}$  and  $(B_{rem} = 0 \text{ or } \text{NOEVIDENCE}(e_i, C))$  then
12:         $a_i \leftarrow \text{DOWNGRADE}(\text{REPLACE}, \text{NO-OP}, \mathcal{P})$ 
13:      end if
14:    else
15:       $a_i \leftarrow z_i$ 
16:    end if
17:     $(D^{(t+1)}, c_i, \tau_i) \leftarrow \text{SIMULATEORREPLAY}(D^{(t)}, e_i, a_i, \Omega_{a_i}, C)$ 
18:     $r_i \leftarrow \text{SHAPEREWARD}(s_i, a_i, c_i, q)$  and store transition in  $\mathcal{B}_1$  or
      $\mathcal{B}_2$ 
19:    Periodically add proxy reward  $\Delta U(M, T, D^{(t+1)}) - \lambda c_i$ 
20:    Update  $Q_1, Q_2$  from replay buffers and softly update  $\bar{Q}_1, \bar{Q}_2$ 
21:  end for
22:  Add terminal reward from  $U(M, T, D^{(T)})$ , retention, and spent budget
23:  Save  $\theta$  if validation utility improves, breaking ties by lower human
   cost
24: end for
25: return best historical allocator  $\pi_\theta$ 
```

candidate values. The executor first separates single-scope operators from multi-attribute operators. Single-scope operators can run as a parallel preprocessing layer. Multi-attribute operators form a dependency graph. If $A_{t_i} \cap A_{s_j} \neq \emptyset$, then the output of o_i can affect the evidence used by o_j . If two operators share the same target scope, their repairs can conflict. Cycles are merged into a block, independent blocks are scheduled by topological level, and operators inside a block are ordered by their current weights.

Within a repair block, the executor generates candidate operations rather than immediately editing the table. For low-quality tuples, it collects source-attribute values as predicates that locate the affected records. From high-quality tuples, cached repaired data, or external knowledge, it collects candidate target values. A candidate repair operation has the form $\ell = (\text{REPAIR}, p, A_t, v', o, c_\ell)$, where p is the predicate scope, v' is the proposed value, o is the originating operator, and c_ℓ is its cost. For example, if an action selects REPAIR for a violation involving $A_s \rightarrow A_t$, dependency and context signals may propose different values for A_t under the same predicate p . The search procedure evaluates candidates by the weighted residual detector score plus modification cost. It applies the best candidate only if it reduces the block objective, updates the data and quality scores, removes redundant candidates with the same predicate or target, and stops when no candidate further improves the block. This gives the REPAIR action a concrete compilation path

Algorithm 2 Inference-Time Action-Workflow Execution

Require: Demand q , data D , units E , allocator π_θ , operator registries Ω , validator (M, U)

Ensure: Prepared data D_ρ , workflow W_ρ , provenance trace τ_ρ

```
1:  $D_{sel} \leftarrow D$ ,  $W_{sel} \leftarrow \emptyset$ , and  $\tau_{sel} \leftarrow \emptyset$ 
2:  $D_{full} \leftarrow \text{APPLYALLOPERATORS}(D, \Omega)$  and  $D_{noop} \leftarrow D$ 
3: for each preparation unit  $e_i \in E$  do
4:    $s_i \leftarrow \text{ENCODE}(D_{sel}, e_i, q, S, B, \tau_{sel})$ 
5:    $a_i \leftarrow \pi_\theta(e_i, s_i, q)$ 
6:    $G_{a_i} \leftarrow \text{BUILDOPERATORGRAPH}(\Omega_{a_i}, D_{sel}, e_i, \tau_{sel})$ 
7:   if  $a_i = \text{NO-OP}$  then
8:     Record pass-through decision for  $e_i$  in  $\tau_{sel}$ 
9:   else
10:     $\mathcal{K}_i \leftarrow \text{GENERATECANDIDATES}(G_{a_i}, D_{sel}, e_i)$ 
11:     $O_i \leftarrow \text{SEARCHOPERATIONS}(\mathcal{K}_i, G_{a_i}, q)$ 
12:     $(D_{sel}, W_{sel}, \tau_{sel}) \leftarrow \text{COMMITIFIMPROVES}(O_i, D_{sel}, W_{sel}, \tau_{sel})$ 
13:    Update operator weights from accepted, rejected, and conflicting
     candidates
14:   end if
15: end for
16:  $u_{rollback} \leftarrow U(M, T, D_{noop})$  and  $u_{selective} \leftarrow U(M, T, D_{sel})$ 
17:  $u_{fullops} \leftarrow U(M, T, D_{full})$ 
18:  $\rho \leftarrow \arg \max_{\rho \in \{\text{rollback}, \text{selective}, \text{fullops}\}} u_\rho$ 
19: return data, workflow, and trace associated with  $\rho$ 
```

through operator association, block construction, operator ordering, candidate generation, repair search, and weight update. The committed output is not a cleaner name alone. It is an operation such as assigning a repaired value to a target attribute or removing a tuple under a recorded predicate.

The other action families use the same grounding interface with different operator networks. REPLACE orders lower-cost estimators, such as dependency-based lookup, neighborhood estimation, cached cleaned-value lookup, or domain-constrained imputation, when exact repair is not demanded or not available. DELETE materializes a row- or cell-scope filtering operation subject to retention constraints, so that one uncertain signal is not enough to remove a row. NO-OP records an intentional preservation decision rather than silently skipping the unit. The current evaluation instantiates these four cleaning actions. The important interface property is that execution is action-specific and workflow-oriented rather than cleaner-specific.

4.4 Step 4: Provenance and Feedback

The fourth step records the action, workflow, operator, and operation trace and feeds the observed utility back to both the allocator and the operator networks. The trace stores the preparation-unit identifier, selected action family, stage decisions, ordered operator workflow, operation identifier ℓ , source scope, target scope, predicate, dirty value, candidate value, final value, input data version, output data version, execution cost, and validation delta. In the current implementation this information is materialized as a decision log, a deferred repair plan, a repair-source log, an operator execution trace, and a run-level metric file.

For a repair action, provenance is produced at four granularities. The action record explains why the unit entered REPAIR rather than NO-OP, DELETE, or REPLACE. The workflow record stores the ordered operators and the dependency block that produced the

candidates. The operator record identifies the source-target dependency, such as $A_s \rightarrow A_t$, and the signal that generated a candidate. The operation record stores the concrete ℓ , its predicate scope, target attribute, candidate value, originating operator, and status. The status indicates whether the candidate was committed, pruned as redundant, rejected because it did not reduce the block objective, or overwritten by a higher-weight candidate. The operation-level record makes multi-cleaner execution auditable by exposing the final value, selected action family, operator order, materialized data operation, and discarded competing candidates.

Feedback is used at two levels. At the action level, downstream validation supplies the reward signal that makes the allocator prefer high-utility actions under the current task. If the prepared candidate lowers validation utility relative to the reference data, rollback selects No-op at the dataset level even when many cell-level edits look plausible. If a candidate improves the metric, the corresponding action decisions receive positive evidence. At the operator level, the same trace updates the priority of signals inside an action family. Operators whose candidate fixes survive conflict resolution and improve the block or downstream objective receive higher weights in later repair blocks. Operators that produce overwritten, conflicting, or harmful candidates are down-weighted or skipped in later candidate generation. Feedback is not only a final accept/reject gate. It changes both the action preference learned by the policy and the operator preference used by the executor when a later unit is assigned to the same action family.

This mechanism is the governance layer of DDPAGENT. The system outputs a data version together with a reproducible action, workflow, operator, and operation history and a recoverable feedback path. The same trace can support audit, rollback, and reuse. If a future task has a similar demand tuple, the allocator can reuse action-level evidence, and the executor can initialize operator ordering and weights from previously successful action networks.

4.5 Governance Decisions Exposed by the Loop

The previous four steps define the execution mechanism. They also expose the decisions that a data-governance agent must report to users and to later workflow runs. These decisions are not extra assumptions. They are the operational consequences of separating task demand, action allocation, operator grounding, and validation feedback.

Demand control is separated from operator execution. Action allocation and operator execution should not be collapsed. Data cleaning research has produced many specialized repair, imputation, filtering, and consistency-checking operators. A scalable preparation agent should keep these methods as reusable operators and learn a control layer that decides whether the current task demands them. This also avoids an important failure mode of tool-using agents. Once the agent chooses a concrete cleaner too early, it has already committed to the cleaner’s objective rather than the downstream task objective.

Preservation and full execution are accountable regimes. No-op should be modeled as an explicit action. In an operator-only pipeline, preservation is usually treated as the absence of a decision. In a governance agent, it is a positive decision. The system may preserve a suspicious value because the detector is uncertain, the

model is tolerant, or the sample carries useful distributional information. FULLOPS has the symmetric meaning at the workflow level. It records that the task benefits from broad operator application, so selective pruning is unnecessary or harmful for that demand. These two regimes define the conservative and intensive ends of the same governance space. Neither should be interpreted as an external baseline that defeats the agent.

Action families define the extension boundary. The current instantiation uses NO-OP, REPAIR, DELETE, and REPLACE because the experiments focus on cleaning. The key requirement is not that every cleaner be redesigned, but that each operator exposes its scope, cost, output data version, and validation feedback. The same boundary can host non-cleaning preparation families only after they are made explicit in the demand tuple. For instance, a future augmentation family would need its own unit definition, candidate generator, label or feature scope, cost, and provenance fields before the allocator can choose it. Under this view, traditional cleaning methods are not superseded by the agent. They become governed operators under task-aware action families.

Preparation is evaluated as a governed workflow. Cell-level repair accuracy remains useful for diagnosing an operator, but it is insufficient for an autonomous preparation agent. The system should be judged by downstream utility, execution cost, human-label cost, traceability, and the ability to recover from harmful interventions. This boundary motivates the native-error, injected-error, and feedback analyses in Section 5. It also explains why the evaluation includes diagnostic controls such as full repair and oracle deletion. These controls test whether aggressive correctness-oriented interventions help the downstream workflow.

5 EXPERIMENTAL EVALUATION

The current implementation instantiates DDPAGENT for tabular cleaning. Candidate errors are obtained from available detectors and cached cleaning outputs. The action allocator is trained without additional human labels and predicts action families for the detected issues. The executor compiles selected actions into ordered cleaning workflows, emits data operations, and produces a prepared table. Downstream validation then evaluates the model on a fixed split and selects among conservative rollback, selective execution, and full operator execution according to task utility. The evaluation focuses on whether the control-and-execution design identifies the demanded governance regime, rather than cell-level repair accuracy alone.

We ask three questions. **RQ1** asks whether the final governance output competes with fixed cleaning methods on native errors. **RQ2** asks whether the same control loop remains useful under controlled mixed-error injection rather than only on the native data. **RQ3** asks when the agent should preserve, selectively intervene, or fully apply operators, and whether action traces explain these regimes.

5.1 Experimental Setup

The evaluation uses five datasets with native errors and the same five datasets under mixed-error injection. Table 1 lists the task settings. Following standard cleaning reports, C% and R% denote cell-level and record-level error rates on the task rows. M, T, F, and V abbreviate missing values, typos/outliers, format inconsistencies,

Table 1: Summary of datasets and downstream tasks. C and R denote classification and regression.

Data	Rows	Col.	C%	R%	Err.	Task
Beers	2.4K	10	13.94	100.00	M/F/V	C-style
Flights	1.1K	7	15.45	56.42	M/F/V	C-delay
Hospitals	1.0K	19	2.68	40.70	T/V	C-measure
Rayyan	642	11	10.25	84.58	M/T/F/V	C-lang.
Tax-10K	10K	15	0.11	1.65	T/F/V	R-rate

and attribute-dependency violations. The large Tax case is evaluated on a clustered 10K subset rather than a random sample, so records sharing key-like structure remain together. The Hospitals target is measure-code classification. This avoids the saturated ownership-label split and gives cleaning actions a measurable downstream effect. The reported experiments use a singleton downstream model per dataset. In this setting, the candidate-model interface in Section 3 is exercised as fixed-model task demand rather than as model selection.

We evaluate fixed cleaning baselines, deployable DDPAGENT regimes, and oracle diagnostic references. For each fixed cleaner, we train and evaluate the same downstream model directly on the cleaned table produced by that method. This protocol compares data preparation outputs rather than internal cell-level objectives. We also report two internal governance regimes of DDPAGENT. The No-op regime preserves the dirty input and is selected when validation indicates limited measurable data-side gain. The FULLOPS regime bypasses selective pruning, directly takes the complete all-operator workflow output from the execution substrate, and trains the downstream model on that full output. FULLOPS answers when the task demands broad operator application, while selective DDPAGENT tests whether task-conditioned action allocation should commit only the operations emitted by selected workflows. Oracle deletion removes rows containing known true errors and tests whether aggressive authenticity improvement helps after sample loss. Full ground-truth repair replaces all known erroneous cells with clean values and tests whether maximum cell-level correctness is aligned with task utility. These oracle diagnostics use clean-reference information that a practical system does not have at deployment time, so they are diagnostic references rather than ordinary cleaning baselines or DDPAGENT operating modes. The reported native setting contains 45 baseline, regime, and reference evaluations. The reported injected setting contains 40 preparation cases and 360 baseline, regime, and reference evaluations.

Table 2 organizes the evaluated methods by the decision they are allowed to make before downstream evaluation. This taxonomy is important for interpreting the results. A fixed cleaner is allowed to output one cleaned table. A DDPAGENT regime is allowed to decide which prepared data version should be committed. An oracle diagnostic is allowed to use clean-reference information that is unavailable at deployment time. The table also records the evidence each method uses, the action or regime it can express, and its role in the evaluation.

For injected data, we follow the mixed-error construction used in data cleaning benchmarks. For each dataset, non-key attributes

and applicable constraints are independently perturbed with typographical or outlier edits, missing values, and rule-violation values. We use eight cell- or rule-level injection rates between 0.25% and 2.0%. Dataset names with * denote arithmetic means over these eight injected settings. No dataset is reported at a single favorable injection level. Since a tuple may contain several attributes and participate in multiple constraints, record-level error rates can be much higher than the nominal cell- or rule-level rate. Table 3 reports the measured cell-level and record-level intensity against the clean reference after applying the same task filtering and subsetting as the evaluation. Across the 40 reported injected task tables, the average cell-level error rate is 1.58%, the average record-level error rate is 17.78%, and the hardest setting reaches 50.90% erroneous rows. The resulting 40 cases require the allocator to avoid unnecessary changes under mild corruption and select demanded interventions when mixed errors affect downstream utility.

5.2 Performance on Native Errors

Table 4 compares the final DDPAGENT governance output with fixed cleaning outputs. The DDPAGENT column reports the validation-selected regime among conservative No-op rollback, selective action allocation, and FULLOPS execution. Table 5 separates these regimes. The values are downstream task utilities after training on each prepared table. Accuracy is used for classification and R^2 for Tax regression, and higher is better. Across the table, DDPAGENT has the highest mean utility among the named fixed cleaners (0.5372) and is ranked first or second on almost every row, with Hospitals* as the only third-place case. The best and second-best values in each row are marked in **red** and **blue**, respectively, with ties included.

On native errors, DDPAGENT is best among fixed cleaned outputs on Beers, Flights, and Hospitals, and second on Rayyan and Tax-10K. The gains over the strongest fixed cleaner are 0.0122 on Beers, 0.0239 on Flights, and 0.0167 on Hospitals, corresponding to relative improvements of 6.8%, 2.5%, and 2.0%. The two second-place cases are also informative. DDPAGENT is only 0.0006 below BigDancing on Tax-10K, but 0.0279 below Holistic on Rayyan, where a fixed constraint-oriented output is better aligned with the language-classification target. This pattern is more useful than a simple win count. On Flights, fixed cleaning outputs vary widely. Baran remains close to DDPAGENT, but HoloClean and Horizon reduce downstream utility by 0.7400 and 0.1884 relative to DDPAGENT. A high-quality cell repair method is not automatically aligned with the task metric.

Finding 1. Native preparation is not monotone in cleaning strength. Native errors expose a non-monotonic preparation effect. Running a stronger cleaner or more complete workflow does not necessarily improve downstream utility. In Table 5, the selective policy improves over FULLOPS by 0.1194 on Flights, 0.0313 on Hospitals, and 0.0267 on Rayyan, while tying it on Beers and Tax-10K. The same table shows that DDPAGENT matches No-op on Flights, Hospitals, and Rayyan, indicating that validation sometimes prefers preservation after evaluating a candidate execution. These outcomes are operating-regime decisions, not failures. Matching No-op means that the system found no measurable benefit from committing edits under the current split and metric. Matching or

Table 2: Taxonomy of evaluated methods by decision boundary. “Boundary” describes what the method is allowed to decide. “Coverage” lists the action family or workflow regime it can express. “Oracle” marks clean-reference access.

Family	Method	Boundary	Evidence	Coverage	Evaluation role	Oracle
Fixed cleaner	Baran [14]	cell repair	transferred cell context	repair	contextual repair base-line	No
Fixed cleaner	BigDancing [10]	cleansing workflow	rule/operator execution	repair/filter	workflow baseline	No
Fixed cleaner	Holistic [3]	repair set	denial constraints	repair	constraint baseline	No
Fixed cleaner	HoloClean [21]	probabilistic repair	constraints and co-occurrence	repair	probabilistic baseline	No
Fixed cleaner	Horizon [23]	dependency plan	discovered dependencies	repair	dependency baseline	No
DDP regime	DDPAGENT	unit action plus data-version regime	downstream demand and workflow trace	multi-action	final governed output	No
DDP regime	No-op	committed data version	dirty input and validation feedback	rollback	conservative regime	No
DDP regime	FULLOps	committed data version	all-operator workflow and validation feedback	full execute	intensive regime	No
Oracle diag.	OracleDel	true-error row deletion	clean-reference error rows	delete	deletion diagnostic	Yes
Oracle diag.	GTRepair	true-cell repair	clean-reference erroneous cells	repair	correctness diagnostic	Yes

Table 3: Cell-level and record-level intensity of the injected task tables. Each row summarizes the eight injected settings used for that dataset’s aggregate injected result. Tax is measured on the clustered 10K subset.

Data	Ver.	Avg. C%	Range C%	Avg. R%	Range R%
Beers	8	1.30	0.62 to 1.98	12.50	6.18 to 17.93
Flights	8	1.89	0.44 to 3.35	11.14	3.03 to 19.23
Hospitals	8	1.95	0.41 to 3.47	31.00	8.30 to 50.90
Rayyan	8	0.88	0.42 to 1.34	9.72	4.90 to 14.20
Tax-10K	8	1.87	0.45 to 3.19	24.55	6.40 to 40.15
Mean	40	1.58	0.41 to 3.47	17.78	3.03 to 50.90

using FULLOps means that full operator application is the demanded regime. The target is not maximum edit distance from the dirty table, nor maximum agreement with one cleaning objective. The target is a task-conditioned intervention policy that can be conservative, selective, or intensive.

5.3 Performance on Injected Errors

The injected aggregate rows in Table 4 use the perturbation protocol and record-level intensities reported in Table 3. They test whether the action allocator remains useful when the error distribution is controlled but heterogeneous. DDPAGENT is best on Beers*, Flights*, and Tax-10K*, with relative gains of 4.9%, 0.8%, and 13.7% over the strongest fixed cleaner on those rows. On Rayyan*, the final system promotes FULLOps and becomes second, 0.0189 below Holistic. On Hospitals*, DDPAGENT is 0.0084 below Holistic and 0.0012 below BigDancing, showing a concrete boundary where a fixed cleaner remains slightly better under the complete all-rate

Table 4: Comparison with fixed baseline cleaning methods. The DDPAGENT column reports the final governance output selected among No-op rollback, selective allocation, and FULLOps execution. When a fixed cleaner does not produce an applicable output for a row, we report the corresponding No-op utility as a conservative fallback. Injected rows follow the aggregation protocol in Section 5. Red and blue mark the best and second-best downstream utilities in each row, with ties included.

Data	DDPAGENT	Baran	BigD.	Hol.	HoloC.	Horiz.
Beers	0.1929	0.1807	0.1607	0.1526	0.1511	0.1540
Flights	0.9854	0.9615	0.9297	0.9312	0.2454	0.7970
Hospitals	0.8665	0.8392	0.8075	0.8002	0.8498	0.8457
Rayyan	0.1538	0.1183	0.1205	0.1817	0.0685	0.0825
Tax-10K	0.5756	0.5753	0.5762	0.5751	0.4720	0.3294
Beers*	0.1821	0.1728	0.1720	0.1736	0.1724	0.1718
Flights*	1.0000	0.9357	0.9910	0.9923	0.2706	0.9099
Hospitals*	0.8571	0.8532	0.8583	0.8655	0.8460	0.7858
Rayyan*	0.1700	0.1489	0.1214	0.1889	0.0395	0.1377
Tax-10K*	0.3883	0.3414	-6.5793	-1.6333	-5.6384	0.1318

average. The Tax-10K* row is particularly important because several fixed cleaned outputs degrade the regression utility sharply, while DDPAGENT preserves a positive utility close to the stable references. This supports the demand-driven view. A governance agent is useful when it edits helpful cells and when it avoids committing to an operator whose repairs are harmful for the current task.

The injected rows also show the boundary of the current implementation. Holistic is strongest on Hospitals* and Rayyan*, and FULLOps is strongest among the reported DDPAGENT regimes on

Rayyan* in Table 5. When one fixed cleaning workflow or the complete all-operator workflow is consistently well aligned with a task and error distribution, the executor should be able to reuse it without forcing selective pruning. The contribution of DDPAGENT is the control interface that can select, skip, roll back, or escalate to full operator execution under downstream feedback rather than claiming that one learned selective policy dominates every fixed cleaner.

Finding 2. Mixed errors make operator choice a governance problem. Mixed-error injection shows why a governance agent needs to handle heterogeneous operator effects. Across the 40 injected settings, DDPAGENT reaches or exceeds the best fixed cleaning output in 21 cases, improves over No-OP in 17 cases, and triggers rollback in 18 cases. The action traces explain this split. On injected Beers, the allocator uses a repair/replace-heavy mix with 44.7% REPAIR and 48.6% REPLACE. On injected Rayyan, REPLACE dominates at 60.3%. On injected Tax, the policy shifts toward 37.1% No-OP and 36.5% DELETE. Mixed errors do not demand a uniform cleaning intensity. Some tasks need value-level correction, some need low-cost replacement, and some benefit from preservation or removal. This result supports the action-family abstraction in Section 4. Mixed errors should not force the system to trust a single cleaner globally, because different corruption patterns and downstream tasks demand different intervention families.

5.4 Action Allocation and Feedback Analysis

Table 5 separates three DDPAGENT governance regimes from diagnostic controls. No-OP is the conservative rollback regime. FULLOPS is the intensive regime that commits the complete all-operator workflow output. DDPAGENT-SEL reports the selective action-allocation branch before No-OP rollback or FULLOPS promotion. OracleDel deletes rows containing known true errors, and GTRepair repairs all known erroneous cells with clean values. These two use clean-reference information and are diagnostic controls rather than deployable baselines. When OracleDel has no applicable deletion output for a row, the table reports the No-OP utility. The table is not a fixed-cleaner comparison. It explains when the data governance agent should preserve, selectively intervene, fully apply operators, delete by oracle knowledge, or repair by oracle knowledge.

Finding 3. Feedback makes preparation recoverable rather than one-shot. Table 5 shows that No-OP is a meaningful governance regime rather than a weak baseline. Across the ten native/injected aggregate rows, DDPAGENT-SEL is never below No-OP, improves over it on six rows, and ties it on four. A tie with No-OP is still informative. It means that validation chose or matched the conservative outcome because data-side intervention did not yield measurable utility gain. The selective branch also wins or ties against FULLOPS on nine rows, with an average utility gain of 0.0254. The remaining case, Rayyan*, is a regime where the complete all-operator workflow is better than selective pruning. GTRepair is not an absolute upper bound on downstream utility either. The final DDPAGENT output matches or exceeds GTRepair on most rows, including Hospitals and Hospitals*, where full repair is worse than selective preparation. These comparisons separate governance regimes that are often conflated in cleaning papers. Preserving the dirty input, selectively applying demanded actions,

Table 5: DDPAGENT operating regimes and diagnostic comparison. No-OP is the conservative rollback regime, FULLOPS is the full operator-execution regime, and Sel. is the selective action-allocation branch before rollback or FULLOPS promotion. OracleDel and GTRepair use clean-reference information and are included only to interpret action choices. Red and blue mark the best and second-best values in each row.

Data	OracleDel	FULLOPS	Sel.	GTRepair	No-OP
Beers	0.1511	0.1929	0.1929	0.1901	0.1511
Flights	0.9391	0.8660	0.9854	1.0000	0.9854
Hospitals	0.7907	0.8352	0.8665	0.8456	0.8665
Rayyan	0.1282	0.1271	0.1538	0.1540	0.1538
Tax-10K	0.5755	0.5756	0.5756	0.5755	0.5751
Beers*	0.1724	0.1790	0.1821	0.1773	0.1724
Flights*	0.9968	0.9027	1.0000	1.0000	1.0000
Hospitals*	0.8125	0.8413	0.8571	0.8510	0.8564
Rayyan*	0.1104	0.1700	0.1278	0.1414	0.1241
Tax-10K*	0.3878	0.3861	0.3883	0.3881	0.3882

fully executing the operator pool, and repairing all known errors are distinct outcomes, and the desirable one is task-dependent.

The action logs add a second layer of evidence. In the native runs, the candidate policy is replacement-heavy overall. It assigns 59.4% of actions to REPLACE, 23.8% to REPAIR, 9.0% to DELETE, and 7.8% to No-OP. This aggregate hides strong dataset dependence. Native Beers and Flights are dominated by REPLACE (95.2% and 74.2%). Native Hospitals is repair-heavy (67.3% REPAIR), while native Rayyan splits between No-OP and DELETE (47.6% and 52.4%). Native Tax is almost entirely deletion in the candidate trace (99.6% DELETE), which matches the intuition that a sparse set of structurally problematic records can be more harmful than individual cell noise for the regression target. In the injected setting, the overall mix shifts to 40.4% REPAIR, 34.8% REPLACE, 19.7% No-OP, and 5.1% DELETE. The governance point is that action intensity is data- and task-conditioned. The same framework can become conservative, repair-oriented, replacement-oriented, or deletion-oriented depending on downstream feedback.

Figure 3 summarizes the same behavior across the injected settings. Panel (a) counts settings where DDPAGENT reaches or exceeds the best fixed cleaned output, improves over No-OP, or rolls back a harmful candidate. These quantities correspond to the responsibilities of the agent. It should use reusable operators when they help, stay conservative when data-side gain is limited, and recover when a plausible execution is harmful. Panel (b) shows that the recovery behavior is not produced by one fixed action template. Beers and Rayyan allocate nearly half or more of their decisions to REPLACE. Flights and Hospitals are repair-heavy. Tax-10K splits between No-OP and DELETE. The complementary FULLOPS results in Table 5 add the intensive regime. When full execution is best, the data governance conclusion is that the current task needs broad operator application rather than selective intervention. A harmful repair is not only a bad output. It is a signal that can update future action preferences and operator weights. DDPAGENT treats every

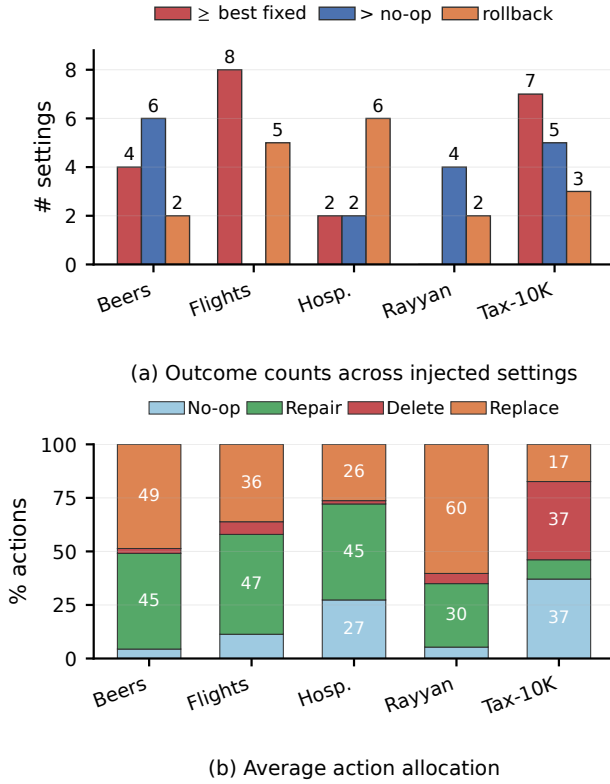


Figure 3: Injected-case behavior of DDPAGENT across the five reported datasets. Panel (a) counts, out of eight injected settings per dataset, when DDPAGENT reaches or exceeds the best fixed cleaning output, improves over No-op, or rolls back a harmful candidate. Panel (b) reports the average action allocation from the decision logs over the same eight settings.

prepared table as a candidate data version, not as an irreversible endpoint.

6 CONCLUSION

DDPAGENT operationalizes demand-driven data preparation as a controller and executor loop. The controller decides which action family or workflow regime is demanded by the downstream task, and the executor grounds that decision as ordered operator workflows and materialized data operations. The cleaning instance demonstrates this structure with four action families and three deployment outcomes, namely conservative No-op rollback, selective action allocation, and intensive FULLOPs execution.

The current implementation is deliberately scoped. The evaluated operators are tabular cleaning operators, the downstream model is fixed per dataset, and the reported action families are No-op, REPAIR, DELETE, and REPLACE. The paper does not claim a natural-language planning interface or a benchmark over non-cleaning preparation actions. The framework is still extensible in a precise sense. A new action family, such as data augmentation, can

be added by registering its units, signals, operators, costs, and provenance schema under the same action-indexed executor. Within the evaluated scope, a preparation agent should decide whether the downstream task demands repair, deletion, replacement, broad operator execution, or no intervention at all.

ACKNOWLEDGMENTS

This work is supported by the National Key Research and Development Program of China (2024YFB3309601), the National Natural Science Foundation of China (NSFC) (62572151, 62232005), the National Key Laboratory of Data Space Technology and System (QZQC2026042), the Natural Science Foundation Project of Heilongjiang Province of China (HSF20230095), and the Heilongjiang Provincial Young Talents Support Program for Science and Technology Innovation (2025QNTJ001).

REFERENCES

- [1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting Data Errors: Where are we and what needs to be done? *Proc. VLDB Endow.* 9, 12 (2016), 993–1004. <https://doi.org/10.14778/2994509.2994518>
- [2] Chengliang Chai, Jiabin Liu, Nan Tang, Ju Fan, Dongjing Miao, Jiayi Wang, Yuyu Luo, and Guoliang Li. 2023. GoodCore: Data-effective and Data-efficient Machine Learning through Coreset Selection over Incomplete Data. *Proc. ACM Manag. Data* 1, 2 (2023), 157:1–157:27. <https://doi.org/10.1145/3589302>
- [3] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic Data Cleaning: Putting Violations into Context. In *Proceedings of the 29th IEEE International Conference on Data Engineering*. IEEE Computer Society, Brisbane, Australia, 458–469. <https://doi.org/10.1109/ICDE.2013.6544847>
- [4] Eduardo Souza dos Reis, Mohamed Abdelaal, and Carsten Binnig. 2024. Generalizable Data Cleaning of Tabular Data in Latent Space. *Proc. VLDB Endow.* 17, 13 (2024), 4786–4798. <https://doi.org/10.14778/3704965.3704983>
- [5] Mohamed Y. Eltabakh, Zan Ahmad Naeem, Mohammad Shahmeer Ahmad, Mourad Ouzzani, and Nan Tang. 2024. RetClean: Retrieval-Based Tabular Data Cleaning Using LLMs and Data Lakes. *Proc. VLDB Endow.* 17, 12 (2024), 4421–4424. <https://doi.org/10.14778/3685800.3685890>
- [6] Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6 (2024), 231:1–231:27. <https://doi.org/10.1145/3698831>
- [7] Johannes Jakubik, Michael Vössing, Niklas Kühl, Jannis Walk, and Gerhard Satzger. 2024. Data-Centric Artificial Intelligence. *Bus. Inf. Syst. Eng.* 66, 4 (2024), 507–515. <https://doi.org/10.1007/S12599-024-00857-8>
- [8] Mohammad Hossein Jarrahi, Ali Memariani, and Shion Guha. 2023. The Principles of Data-Centric AI. *Commun. ACM* 66, 8 (2023), 84–92. <https://doi.org/10.1145/3571724>
- [9] Bojan Karlas, Peng Li, Renzhi Wu, Nezihe Merve Gürel, Xu Chu, Wentao Wu, and Ce Zhang. 2020. Nearest Neighbor Classifiers over Incomplete Information: From Certain Answers to Certain Predictions. *Proc. VLDB Endow.* 14, 3 (2020), 255–267. <https://doi.org/10.5555/3430915.3442426>
- [10] Zuhair Khayyat, Ihab F. Ilyas, Alekh Jindal, Samuel Madden, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Si Yin. 2015. BigDancing: A System for Big Data Cleansing. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, Melbourne, Victoria, Australia, 1215–1230. <https://doi.org/10.1145/2723372.2747646>
- [11] Sanjay Krishnan, Michael J. Franklin, Ken Goldberg, and Eugene Wu. 2017. BoostClean: Automated Error Detection and Repair for Machine Learning. *arXiv:1711.01299* <http://arxiv.org/abs/1711.01299>
- [12] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: Interactive Data Cleaning For Statistical Modeling. *Proc. VLDB Endow.* 9, 12 (2016), 948–959. <https://doi.org/10.14778/2994509.2994514>
- [13] Andreas Loizou and Dimitrios Tsoumakos. 2025. Chunked Data Shapley: A Scalable Dataset Quality Assessment for Machine Learning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM 2025, Seoul, Republic of Korea, November 10-14, 2025*, Meeyoung Cha, Chanyoung Park, Noseong Park, Carl Yang, Senjuti Basu Roy, Jessie Li, Jaap Kamps, Kijung Shin, Bryan Hooi, and Lifang He (Eds.). ACM, Seoul, Republic of Korea, 1958–1967. <https://doi.org/10.1145/3746252.3761305>
- [14] Mohammad Mahdavi and Ziawasch Abedjan. 2020. Baran: Effective Error Correction via a Unified Context Representation and Transfer Learning. *Proc.*

- VLDB Endow. 13, 11 (2020), 1948–1961. <http://www.vldb.org/pvldb/vol13/p1948-mahdavi.pdf>
- [15] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, Peter Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, Amsterdam, The Netherlands, 865–882. <https://doi.org/10.1145/3299869.3324956>
 - [16] Sedir Mohammed, Felix Naumann, and Hazar Harmouch. 2025. Step-by-Step Data Cleaning Recommendations to Improve ML Prediction Accuracy. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, Alkis Simitsis, Bettina Kemme, Anna Queralt, Oscar Romero, and Petar Jovanovic (Eds.). OpenProceedings.org, Barcelona, Spain, 542–554. <https://doi.org/10.48786/EDBT.2025.43>
 - [17] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proc. VLDB Endow.* 17, 10 (2024), 2617–2630. <https://doi.org/10.14778/3675034.3675051>
 - [18] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Yaoshu Wang, and Jianwei Yin. 2025. ZeroED: Hybrid Zero-Shot Error Detection Through Large Language Model Reasoning. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, Hong Kong, 3126–3139. <https://doi.org/10.1109/ICDE65448.2025.00234>
 - [19] Alireza Pirhadi, Mohammad Hossein Moslemi, Alexander Cloninger, Mostafa Milani, and Babak Salimi. 2024. OTClean: Data Cleaning for Conditional Independence Violations using Optimal Transport. *Proc. ACM Manag. Data* 2, 3 (2024), 160. <https://doi.org/10.1145/3654963>
 - [20] Jianbin Qin, Sifan Huang, Yaoshu Wang, Jing Zhu, Yifan Zhang, Yukai Miao, Rui Mao, Makoto Onizuka, and Chuan Xiao. 2024. BClean: A Bayesian Data Cleaning System. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, Utrecht, The Netherlands, 3407–3420. <https://doi.org/10.1109/ICDE60146.2024.00263>
 - [21] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proc. VLDB Endow.* 10, 11 (2017), 1190–1201. <https://doi.org/10.14778/3137628.3137631>
 - [22] Cédric Renggli, Luka Rimanic, Nezihe Merve Gürel, Bojan Karlas, Wentao Wu, and Ce Zhang. 2021. A Data Quality-Driven View of MLOps. *IEEE Data Eng. Bull.* 44, 1 (2021), 11–23. <http://sites.computer.org/debull/A21mar/p11.pdf>
 - [23] El Kindi Rezig, Mourad Ouzzani, Walid G. Aref, Ahmed K. Elmagarmid, Ahmed R. Mahmood, and Michael Stonebraker. 2021. Horizon: Scalable Dependency-driven Data Cleaning. *Proc. VLDB Endow.* 14, 11 (2021), 2546–2554. <https://doi.org/10.14778/3476249.3476301>
 - [24] Shafaq Siddiqi, Roman Kern, and Matthias Boehm. 2023. SAGA: A Scalable Framework for Optimizing Data Cleaning Pipelines for Machine Learning Applications. *Proc. ACM Manag. Data* 1, 3 (2023), 218:1–218:26. <https://doi.org/10.1145/3617338>
 - [25] Jianwei Wang, Ying Zhang, Kai Wang, Xuemin Lin, and Wenjie Zhang. 2024. Missing Data Imputation with Uncertainty-Driven Network. *Proc. ACM Manag. Data* 2, 3 (2024), 117. <https://doi.org/10.1145/3654920>
 - [26] Mengyi Yan, Yaoshu Wang, Yue Wang, Xiaoye Miao, and Jianxin Li. 2024. GIDCL: A Graph-Enhanced Interpretable Data Cleaning Framework with Large Language Models. *Proc. ACM Manag. Data* 2, 6 (2024), 236:1–236:29. <https://doi.org/10.1145/3698811>
 - [27] Sangdoo Yun, Dongyoon Han, Sanghyuk Chun, Seong Joon Oh, Youngjoon Yoo, and Junsuk Choe. 2019. CutMix: Regularization Strategy to Train Strong Classifiers With Localizable Features. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*. IEEE, Seoul, Korea (South), 6022–6031. <https://doi.org/10.1109/ICCV.2019.00612>
 - [28] Chi Zhang, Huaping Zhong, Kuan Zhang, Chengliang Chai, Rui Wang, Xinlin Zhuang, Tianyi Bai, Jiantao Qiu, Lei Cao, Ju Fan, Ye Yuan, Guoren Wang, and Conghui He. 2025. Harnessing Diversity for Important Data Selection in Pretraining Large Language Models. The Thirteenth International Conference on Learning Representations (ICLR 2025). <https://openreview.net/forum?id=bMC1t7eLRc>
 - [29] Hongyi Zhang, Moustapha Cissé, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. 6th International Conference on Learning Representations (ICLR 2018). <https://openreview.net/forum?id=r1Ddp1-Rb>
 - [30] Jiakuan Zhang, Haitao Yuan, Jianing Si, Nan Jiang, and Shangguang Wang. 2025. Think Twice Before Imputation: Optimizing Data Imputation Order for Machine Learning. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, Hong Kong, 1362–1374. <https://doi.org/10.1109/ICDE65448.2025.00106>